Digitopper Project Tracker

Backend Architecture — CTO Summary

Objective: Build a simple, secure, relational and frontend-ready backend without unnecessary models or abstractions.

1. Technology Stack

Node.js • Express.js • MongoDB • Mongoose • JWT • bcrypt/bcryptjs • REST API • Swagger/OpenAPI

2. Core Architecture

The backend deliberately uses a small set of well-designed models. Business concepts such as permissions, assignments, forms and dependencies are handled within their parent domains rather than becoming separate collections.

Model	Purpose
Employee	Authentication, employee identity and global role
Project	Core project information and lifecycle
ProjectMember	Project-scoped designation and membership
Timeline	One runtime timeline per project
TimelineNode	Stage, substage and nested task hierarchy
Activity	Project-level audit/activity history

3. Database Relationship

Employee → ProjectMember → Project → Timeline → TimelineNode. TimelineNode uses **parentNode** for hierarchy and **assignedTo** → **Employee** for contributor assignment.

4. Roles & Authorization

Global: ADMIN, EMPLOYEE. **Project designations:** PROJECT_MANAGER, CONTRIBUTOR, VIEWER.

Authorization is centralized and evaluated as: authenticated employee → global role → project membership → project designation → resource assignment → permission. PM access is project-scoped; Contributor access is node-scoped; Viewer is read-only.

5. Timeline Design

A fixed 10-stage business workflow is initialized automatically by the backend when a project is created. The static definition lives in **src/config/timeline.js**; runtime data is stored as Timeline and TimelineNode documents.

Stages: Lead & Negotiation → Purchase Order → Proforma Invoice → Project Information & Requirement Gathering → School Onboarding → Execution → Tech + Content Testing → Installation → Training → Project Closure.

Execution supports parallel Hardware / Tech / Content workstreams, Hardware conditional paths, and a dependency gate requiring Tech Ready + Content Ready before Tech + Content Testing.

6. TimelineNode — Key Simplification

One TimelineNode model represents stages, substages and nested substages. It contains status, assignment, formSchema, formData, dependencies and metadata. No separate Stage, SubStage, Assignment, Form or TimelineDependency collections are required for the current scope.

7. Forms & Status

Forms are stored directly on TimelineNode as formSchema + formData and validated by the backend. Core statuses are IN_PROGRESS, COMPLETED, ON_HOLD and CANCELLED. Status transitions and dependency checks are centralized in the service layer.

8. Project Creation Flow

Validate request → Create Project → Create ProjectMember → Create Timeline → Initialize TimelineNodes → Create Activity → Commit transaction. MongoDB transactions are used for important multi-document operations.

9. API & Code Structure

API versioning: **/api/v1/**. Main domains: auth, employees, projects, timeline and activity. Request flow: Route → Middleware → Controller → Service → Model → MongoDB. Controllers remain thin; business logic lives in services.

10. Security & Reliability

JWT authentication, password hashing, Helmet, CORS, rate limiting, input validation, resource-level authorization, safe MongoDB querying, environment-based secrets, centralized errors, structured logging, indexes and transactions.

11. Key Architectural Decision

Smallest clean architecture that correctly supports the Digitopper business. The design prioritizes maintainability, clear relationships, centralized business rules and frontend readiness over excessive abstraction.